IUB

Est. 1975

School Website Management System

SOFTWARE DESIGN DOCUMENT (SDD)

Student Name: Nageena Hakam

Roll No: 4007

Submitted To: Sir Asad Ullah

Date: June 10, 2026

Department: Information Technology

The Islamia University of Bahawalpur

ACADEMIC SUBMISSION | DEPARTMENT OF IT | IUB | 2026

◆ Table of Contents

1 Introduction & Document Purpose

1.1 Purpose

This Software Design Document (SDD) provides the complete technical blueprint for the School Website Management System (SWMS) — a web-based platform developed for the Department of Information Technology, Faculty of Computing, The Islamia University of Bahawalpur. It describes the architectural decisions, data models, user-interface layouts, security measures, behavioural diagrams, and non-functional requirements that govern the system.

1.2 Scope

- Centralised web portal accessible by Administrators, Teachers, Students, and Parents.
- Student registration, academic results, attendance tracking, and notice board.
- Role-Based Access Control (RBAC) ensuring data privacy for each stakeholder.
- Downloadable PDF reports for result cards and attendance summaries.
- Fully responsive design: desktop, tablet, and mobile browsers.
- Secure HTTPS communication with encrypted credentials.
- Modular Laravel MVC architecture enabling easy feature additions.

1.3 Intended Audience

This document is submitted to **Sir Asad Ullah** and academic evaluators of the Department of Information Technology, IUB, as part of the course deliverable by **Nageena Hakam (Roll No 4007)**.

1.4 Document Conventions

This SDD follows the IEEE 1016 standard for Software Design Descriptions. Diagrams include Architecture, ERD, UI Wireframes, Use Case, Sequence, and Activity diagrams, providing comprehensive coverage of system behaviour and structure.

1.5 System Overview

The School Website Management System (SWMS) is designed to digitalise the administrative and academic operations of a school. In a typical school environment, information management involves large volumes of student records, attendance data, examination results, notice dissemination, and staff coordination. Manual paper-based processes are time-consuming, error-prone, and difficult to audit.

SWMS addresses these challenges by providing a centralised, role-driven web portal accessible from any device with a modern browser. Administrators gain full CRUD control over all records. Teachers manage attendance, upload results, and communicate via the notice board. Students and Parents access personalised dashboards showing real-time academic performance, attendance summaries, and official notifications.

1.6 Definitions, Acronyms & Abbreviations

Term / Acronym	Definition
SDD	Software Design Document
SWMS	School Website Management System

Term / Acronym	Definition
RBAC	Role-Based Access Control — permissions assigned per user role
JWT	JSON Web Token — stateless bearer token for authentication
ORM	Object-Relational Mapper — maps DB tables to code objects (Eloquent)
MVC	Model-View-Controller — architectural pattern used by Laravel
3NF	Third Normal Form — database normalisation standard
HTTPS	Hyper-Text Transfer Protocol Secure — TLS-encrypted HTTP
CRUD	Create, Read, Update, Delete — standard data operations
API	Application Programming Interface
CSP	Content Security Policy — browser security header
CSRF	Cross-Site Request Forgery — web attack vector
XSS	Cross-Site Scripting — injection attack vector
GPA	Grade Point Average

2 Application Architecture

2.1 System Overview

SWMS employs a classic **Three-Tier Client-Server Architecture** that cleanly separates presentation, business logic, and data concerns. Each tier communicates only with its adjacent tier, maximising modularity and easing future maintenance or scaling. All external traffic is served over HTTPS / TLS 1.3 via an Nginx reverse-proxy that also handles load balancing.

Tier	Name	Key Components	Responsibility
1	Presentation	HTML5, CSS3, JS, Bootstrap 5	Render UI; capture user input
2	Business Logic	PHP 8 / Laravel 10 (MVC)	Validation, rules, API routing, ORM
3	Data	MySQL 8.0 + Redis Cache	Persistent storage, caching, backups

2.2 Architecture Diagram

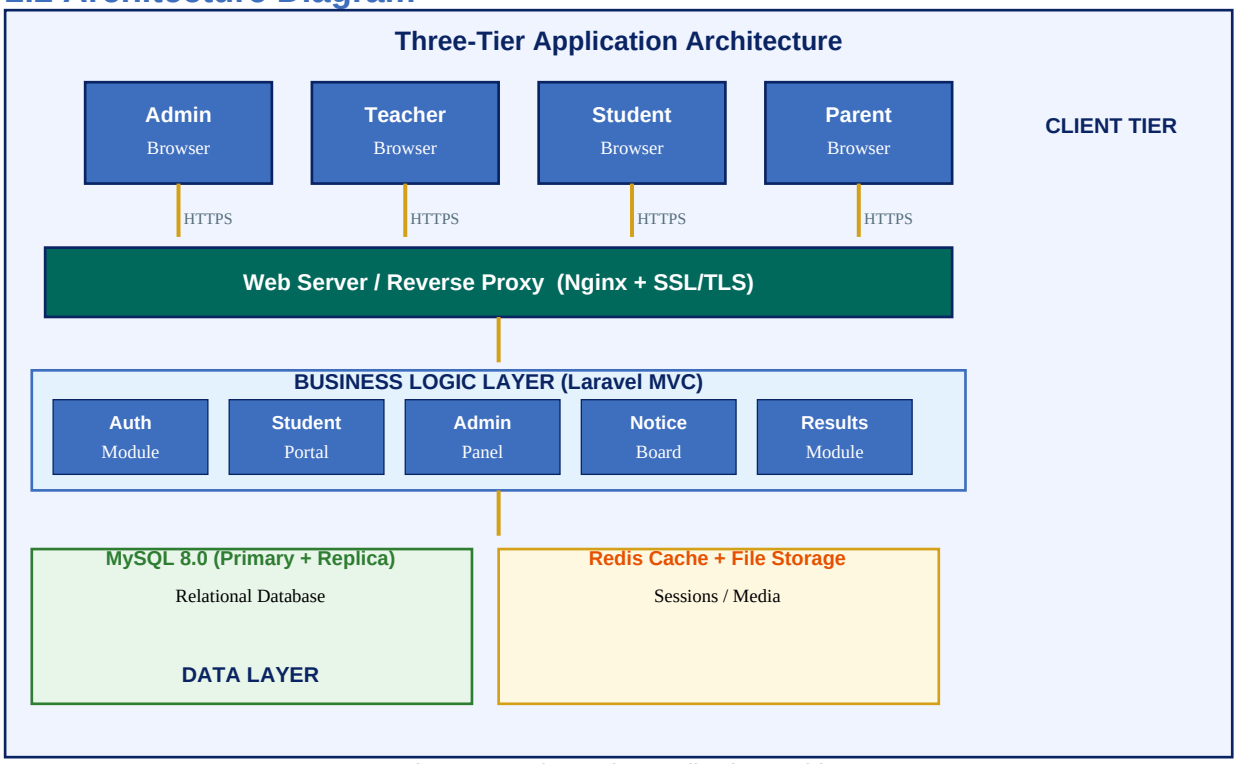

Figure 1 — Three-Tier Application Architecture

2.3 Technology Stack

Category	Technology	Version	Purpose
Frontend	HTML5 / CSS3 / JS	ES2023	Structure, styling, interactivity
CSS Framework	Bootstrap	5.3	Responsive grid & UI components
Backend	PHP	8.2	Server-side scripting
MVC Framework	Laravel	10.x	Routing, ORM (Eloquent), templating

Category	Technology	Version	Purpose
Database	MySQL	8.0	Primary relational data store
Cache	Redis	7.x	Session store, query cache
Web Server	Nginx	1.24	HTTP server, reverse proxy, SSL
Auth	JWT + bcrypt	—	Token auth, password hashing
Version Control	Git / GitHub	2.x	Source code management & CI/CD

2.4 Module Decomposition

The Business Logic Layer is decomposed into five functional modules, each handling a distinct set of responsibilities. Modules communicate through well-defined service interfaces, enabling independent testing, replacement, or enhancement.

Module	Responsibility	Key Classes / Files
Auth Module	Login, logout, token issue/refresh, role detection	AuthController, JWTService, UserModel
Student Portal	Student dashboard, profile management, notifications	StudentController, DashboardView
Admin Panel	Full CRUD for all entities, CSV import, reports	AdminController, ReportService
Notice Board	Create, publish, archive notices; push notifications	NoticeController, NoticeModel
Results Module	Marks entry, grade computation, PDF report generation	ResultController, GradeService

2.5 Deployment Architecture

SWMS is designed for deployment on a LEMP stack (Linux, Nginx, MySQL, PHP). A Docker Compose file bundles all services — Nginx, PHP-FPM, MySQL, and Redis — enabling one-command deployment to any Linux server or cloud VM (AWS EC2, DigitalOcean Droplet). CI/CD is managed via GitHub Actions: push to *main* triggers automated tests, and on success, deploys to the staging server.

Component	Service	Notes
Web Server	Nginx 1.24	SSL termination, gzip, static file caching
App Runtime	PHP-FPM 8.2	FastCGI process manager, 20 workers
Database	MySQL 8.0	Primary + Read Replica for high availability
Cache / Queue	Redis 7.x	Session storage, job queue, query cache
Container	Docker 24.x	docker-compose.yml for all services
CI/CD	GitHub Actions	Automated test, lint, deploy pipeline

3 Data Model Schema

3.1 Entity-Relationship Diagram

The SWMS database is normalised to Third Normal Form (3NF). Seven core entities model all academic and administrative data. Foreign key constraints enforce referential integrity, and indexes on frequently-queried columns (roll_no, student_id, class_id) optimise query performance.

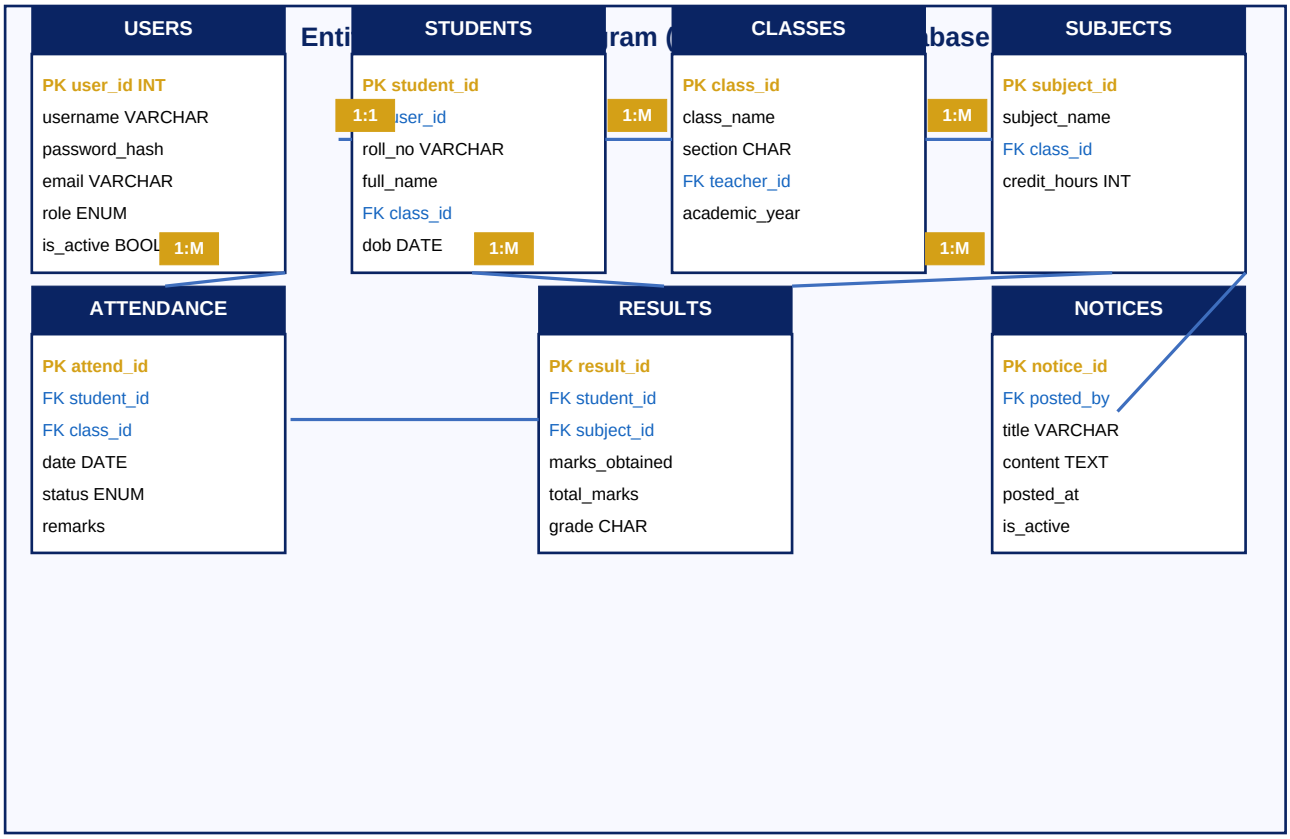

Figure 2 — Entity-Relationship Diagram (ERD) showing 7 entities and their relationships

3.2 Table Definitions (SQL Schema)

USERS Table

```
CREATE TABLE users (
  user_id       INT PRIMARY KEY AUTO_INCREMENT,
  username      VARCHAR(50)  UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  email         VARCHAR(100) UNIQUE NOT NULL,
  role          ENUM('admin','teacher','student','parent') NOT NULL,
  is_active     BOOLEAN DEFAULT TRUE,
  created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

STUDENTS Table

```
CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id    INT NOT NULL,
  roll_no    VARCHAR(20) UNIQUE NOT NULL,
  full_name  VARCHAR(100) NOT NULL,
```

```
  class_id   INT NOT NULL,
  dob        DATE,
  contact    VARCHAR(15),
  FOREIGN KEY (user_id)  REFERENCES users(user_id)    ON DELETE CASCADE,
  FOREIGN KEY (class_id) REFERENCES classes(class_id) ON DELETE RESTRICT
);
```

RESULTS Table

```
CREATE TABLE results (
  result_id      INT PRIMARY KEY AUTO_INCREMENT,
  student_id     INT NOT NULL,
  subject_id     INT NOT NULL,
  marks_obtained FLOAT NOT NULL,
  total_marks    FLOAT DEFAULT 100,
  grade          CHAR(3),
  exam_date      DATE,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);
```

ATTENDANCE Table

```
CREATE TABLE attendance (
  attend_id  INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  class_id   INT NOT NULL,
  date       DATE NOT NULL,
  status     ENUM('Present','Absent','Late','Leave') NOT NULL,
  remarks    VARCHAR(100),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (class_id)   REFERENCES classes(class_id)
);
```

CLASSES Table

```
CREATE TABLE classes (
  class_id      INT PRIMARY KEY AUTO_INCREMENT,
  class_name    VARCHAR(50) NOT NULL,
  section       CHAR(1),
  teacher_id    INT,
  academic_year YEAR NOT NULL,
  FOREIGN KEY (teacher_id) REFERENCES users(user_id)
);
```

SUBJECTS Table

```
CREATE TABLE subjects (
  subject_id   INT PRIMARY KEY AUTO_INCREMENT,
  subject_name VARCHAR(80) NOT NULL,
  class_id     INT NOT NULL,
  credit_hours INT DEFAULT 3,
  FOREIGN KEY (class_id) REFERENCES classes(class_id)
);
```

NOTICES Table

```
CREATE TABLE notices (
  notice_id  INT PRIMARY KEY AUTO_INCREMENT,
  posted_by  INT NOT NULL,
  title      VARCHAR(200) NOT NULL,
  content    TEXT,
  posted_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  is_active  BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (posted_by) REFERENCES users(user_id)
);
```

3.3 Relationships Summary

Entity A	Cardinality	Entity B	Rule / Notes
USERS	1 : 1	STUDENTS	Each user account maps to one student record
CLASSES	1 : M	STUDENTS	One class has many enrolled students
CLASSES	1 : M	SUBJECTS	One class offers multiple subjects
STUDENTS	1 : M	RESULTS	A student has many result records (per subject)
SUBJECTS	1 : M	RESULTS	A subject appears in many result records
STUDENTS	1 : M	ATTENDANCE	Daily attendance tracked per student
USERS	1 : M	NOTICES	Admin/teacher posts multiple notices

4 User Interface Design

4.1 UI Wireframes

The interface follows Material Design principles with a consistent colour scheme, typography hierarchy, and responsive grid. Each role sees a tailored dashboard exposing only relevant features — minimising cognitive load and preventing unauthorised actions.

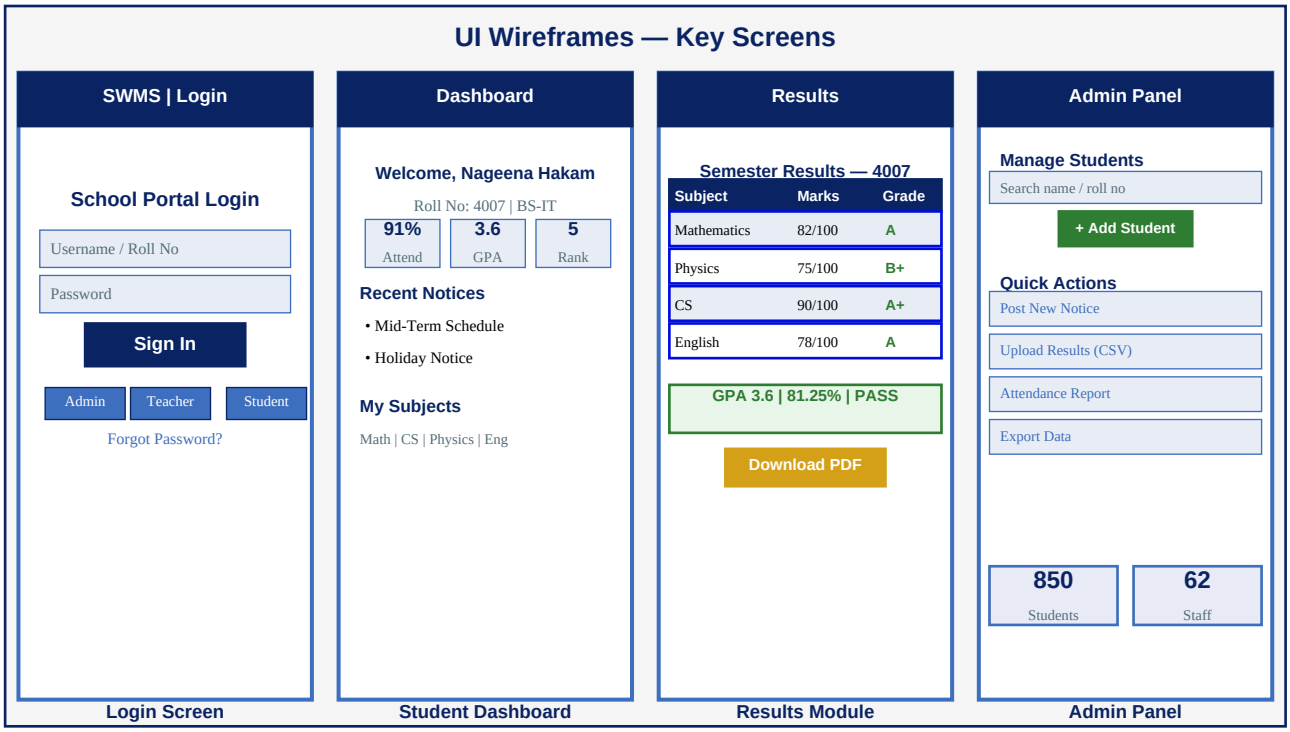

Figure 3 — Key UI screens: Login, Student Dashboard, Results Module, Admin Panel

4.2 Screen Descriptions

Login Screen

- Single entry point for all four roles — credentials determine the redirected dashboard.
- Username (or Roll No) and bcrypt-verified password fields.
- JWT token issued on success; stored in a Secure, HttpOnly cookie.
- Forgot Password triggers a one-time OTP sent to registered email.

Student Dashboard

- Personalised greeting displays student name, roll number, and class.
- KPI cards: Attendance %, cumulative GPA, class rank, and pending items.
- Notice board with unread badge count and timestamp.
- Quick-links to Results, Timetable, Fee Challan, and Profile edit.

Results Module

- Tabular view of all subjects: marks obtained, total marks, letter grade.
- Overall GPA, percentage, and Pass/Fail status computed automatically.
- Downloadable PDF result card with university letterhead.
- Bar chart comparing student marks against class average.

Admin Panel

- CRUD operations for Students, Teachers, Classes, and Subjects.
- Bulk result upload via CSV file with validation error reporting.
- Notice management: post, archive, tag by category (exam / holiday / event).
- Exportable Excel/PDF reports for attendance, results, and fee status.

5 Use Case & Behavioral Diagrams

5.1 Use Case Diagram

The Use Case Diagram identifies the four primary actors — Admin, Teacher, Student, and Parent — and their interactions with core system functionalities within the SWMS boundary. It establishes the scope of system behaviour from the user's perspective.

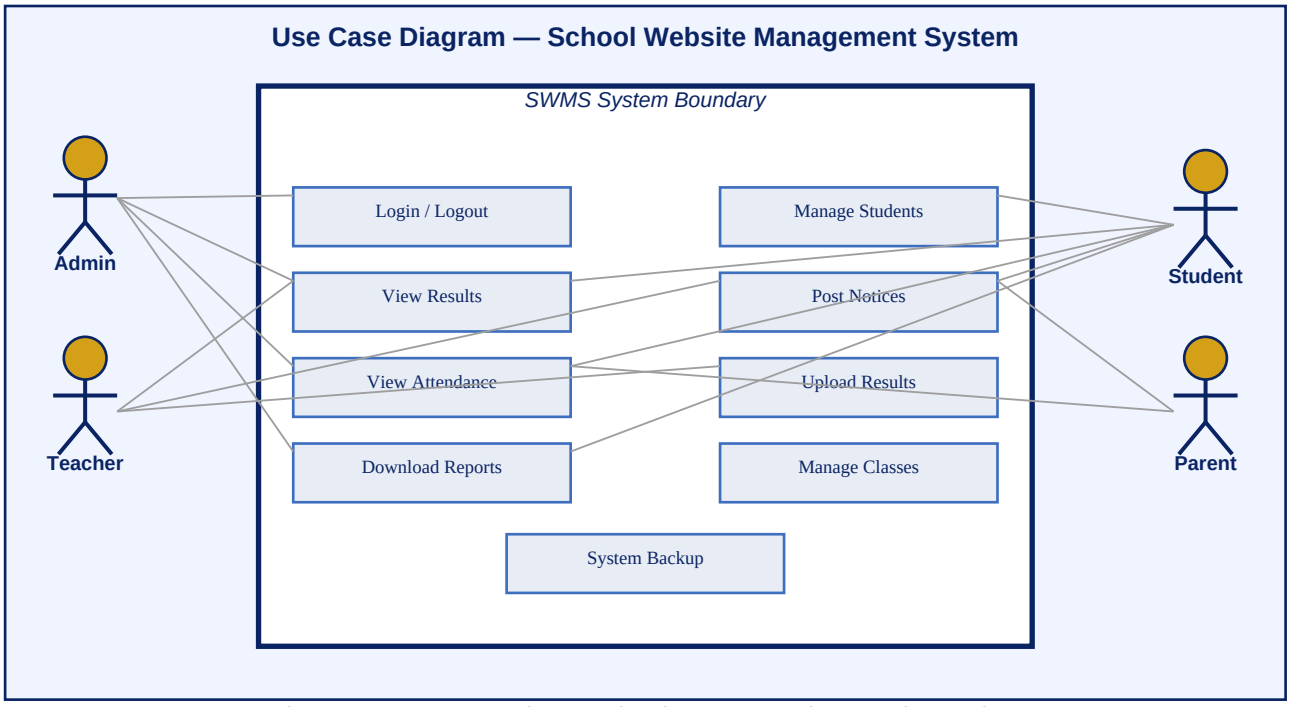

Figure 4 — Use Case Diagram showing actors and system interactions

5.2 Sequence Diagram

The Sequence Diagram illustrates the chronological message flow between system components during the Student Login and Result Retrieval process. It highlights the interaction between the Browser, Auth Module, Laravel Controller, MySQL Database, and Results Module.

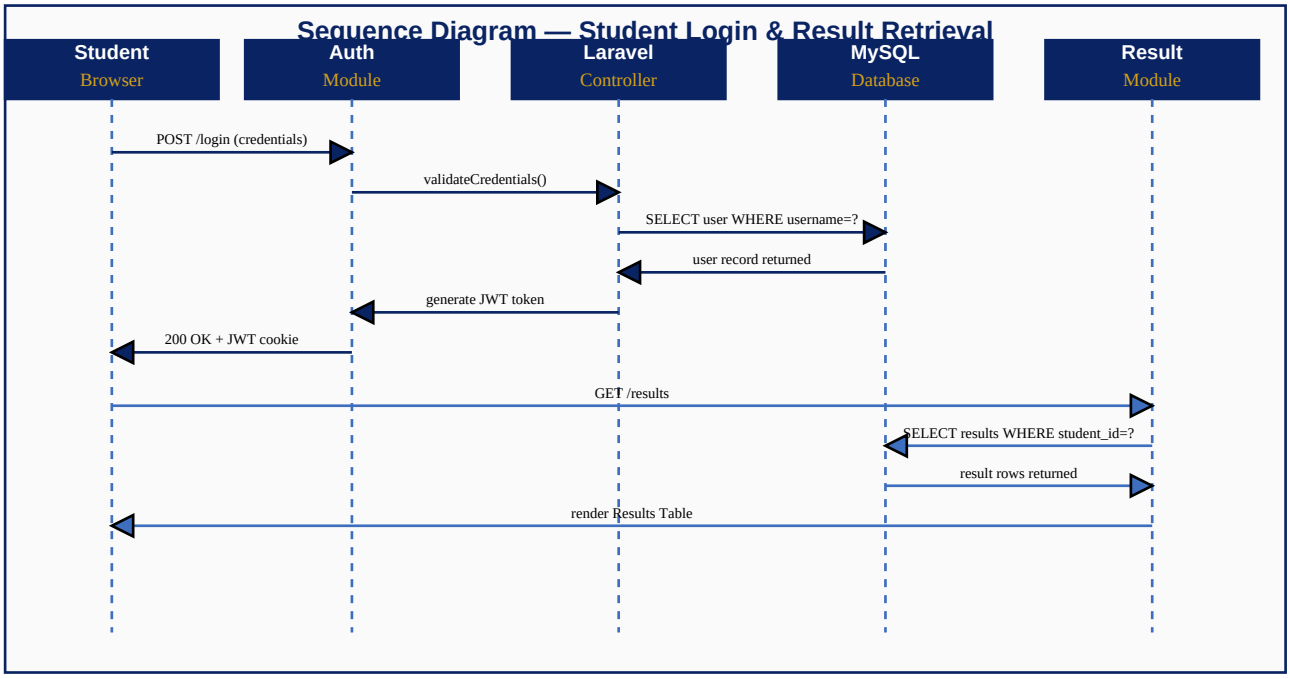

Figure 5 — Sequence Diagram: Student Login and Result Retrieval flow

5.3 Activity Diagram

The Activity Diagram models the workflow followed by an Administrator managing student records within the system. It covers login, navigation to student management, action selection (Add / Edit / Delete), validation, database update, and confirmation steps.

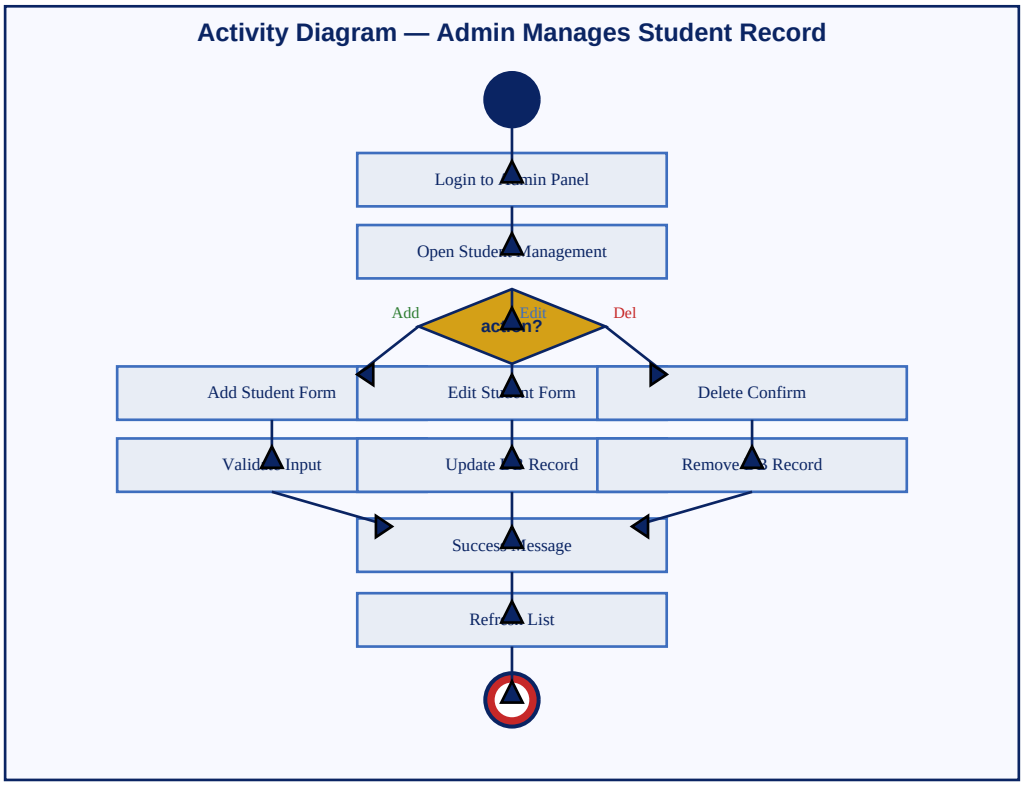

Figure 6 — Activity Diagram: Admin manages student record

5.4 Component Interaction Overview

The following table summarises how each major system component interacts with others, the communication protocol used, and the data exchanged. This provides developers with a quick reference for integration planning.

Source Component	Target Component	Protocol	Data Exchanged
Browser (Student)	Nginx Web Server	HTTPS/TLS	HTTP request with JWT cookie
Nginx	PHP-FPM (Laravel)	FastCGI	Parsed request, headers, body
Laravel Controller	MySQL Database	TCP/3306	SQL queries via Eloquent ORM
Laravel Controller	Redis Cache	TCP/6379	Session tokens, cached queries
Results Module	PDF Generator	In-process	Result data, student info
Auth Module	JWT Library	In-process	User credentials, token claims
Admin Panel	File Storage	Local FS	Uploaded CSVs, exported PDFs
GitHub Actions	Production Server	SSH/SFTP	Compiled assets, migrations

5.5 Error Handling Strategy

SWMS adopts a centralised exception handling approach via Laravel's **Handler.php**. All unhandled exceptions are caught, logged to the application log file (with stack trace and request context), and a user-friendly error page is rendered. API endpoints return structured JSON error responses.

Error Type	HTTP Code	Response	Logged
Validation Failed	422	JSON: field-level error messages	No
Unauthenticated	401	Redirect to Login / JSON 401	No
Forbidden (RBAC)	403	403 Forbidden page / JSON 403	Yes
Resource Not Found	404	404 page / JSON 404	No
Server Error	500	Generic error page / JSON 500	Yes
DB Connection Fail	503	Maintenance page	Yes

5.6 Testing Strategy

A multi-layer testing strategy ensures correctness, performance, and security of SWMS before and after deployment. Tests are automated and integrated into the CI/CD pipeline via GitHub Actions.

Test Type	Tool / Framework	Coverage Target	Trigger
Unit Tests	PHPUnit	Models, Services (>= 80%)	Every commit
Feature Tests	Laravel HTTP Tests	All API endpoints	Every commit
Browser Tests	Laravel Dusk	Critical UI flows	Pre-release
Performance Tests	Apache JMeter	500 concurrent users	Pre-release
Security Scan	OWASP ZAP	OWASP Top-10	Monthly
Accessibility	axe-core	WCAG 2.1 Level AA	Pre-release

Test Coverage Details

- Authentication: valid login, invalid credentials, expired JWT, role-based redirect.
- Student Portal: dashboard data accuracy, GPA computation, attendance percentage.
- Results Module: mark entry, grade calculation, PDF generation, CSV export.
- Admin Panel: CRUD operations, bulk CSV upload, notice publishing.
- Security: SQL injection attempts, XSS payloads, CSRF token validation, rate limiting.
- Performance: 500 concurrent login requests with response time < 2 s.

5.7 Implementation & Development Plan

The project is divided into five development sprints, each two weeks in duration. Milestones are defined per sprint, and progress is tracked via GitHub Projects. Each sprint ends with a functional review and testing cycle.

Sprint	Duration	Milestones / Deliverables
1 — Setup	Week 1-2	Environment setup, DB schema, migrations, user auth (JWT/bcrypt)
2 — Core CRUD	Week 3-4	Student/Class/Subject CRUD, Admin Panel, Role middleware
3 — Academic	Week 5-6	Results module, Attendance module, grade computation
4 — UI Polish	Week 7-8	Dashboard KPIs, responsive design, PDF/CSV export, notices
5 — Testing	Week 9-10	Unit/Feature/Browser tests, performance testing, bug fixes, deployment

5.8 Risks & Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
DB performance under load	Medium	High	Redis caching, read replica, indexed queries
JWT token theft	Low	High	HttpOnly cookies, short TTL, IP binding
CSV import data corruption	Medium	Medium	Row-level validation, rollback on error
Third-party library CVEs	Medium	Medium	Automated dependency scanning via Dependabot
Server downtime	Low	High	Replica DB, automated health checks, alerts
Data loss	Low	Critical	Daily automated backups + offsite storage

6 Security Design

Security is addressed at every tier. The principle of **Defence in Depth** ensures that no single control failure leads to full compromise. All passwords are stored as bcrypt hashes (cost factor 12); plain-text credentials are never logged or stored.

Security Control	Mechanism / Tool	Scope
Authentication	JWT Bearer tokens (24h TTL) + bcrypt	All users
Authorisation	RBAC middleware on every route	All endpoints
Transport Security	TLS 1.3 / HTTPS enforced by Nginx	Client <-> Server
SQL Injection	Eloquent ORM prepared statements	All DB queries
XSS Prevention	Blade templating auto-escaping + CSP header	All HTML output
CSRF Protection	Laravel CSRF token on all state-changing requests	All forms/POST
Session Hardening	Secure, HttpOnly, SameSite=Strict cookies	Session tokens
Rate Limiting	Nginx + Laravel throttle (60 req/min)	Login & API
Audit Logging	All admin actions logged: user, IP, timestamp	Admin actions
Data Backup	Daily automated MySQL dumps + offsite copy	All data

7 Non-Functional Requirements

Attribute	Requirement / Target
Performance	Page load <= 2 s; REST API response <= 300 ms (95th percentile)
Availability	99.5% uptime SLA; scheduled maintenance windows < 2 h/month
Scalability	Support 5,000+ concurrent users; horizontal scaling via load balancer
Usability	WCAG 2.1 Level AA; responsive on 320 px - 1920 px viewports
Maintainability	MVC separation; PHPDoc comments; unit-test coverage >= 80%
Reliability	Daily DB backups; point-in-time recovery; automated health checks
Portability	Runs on any LAMP/LEMP stack; Docker-compose deployment supported
Browser Support	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+, mobile Safari/Chrome
Internationalisation	English (default); Urdu locale ready via Laravel Lang package
Security Compliance	OWASP Top-10 mitigations applied across all tiers

8 Conclusion & Sign-Off

This Software Design Document provides a rigorous, professionally structured design baseline for the **School Website Management System**. The Three-Tier Architecture enforces clean separation of concerns and positions the system for future scaling. The 3NF-normalised relational schema with seven core entities covers every academic workflow from enrolment to result publication while maintaining full referential integrity.

The role-based user interface surfaces only contextually relevant features to each stakeholder, reducing training overhead and the risk of accidental data mutation. Security is layered across transport, application, and data tiers following the Defence in Depth principle, with bcrypt credential hashing, JWT session management, CSRF/XSS mitigation, and comprehensive audit logging.

Behavioural diagrams — Use Case, Sequence, and Activity — provide a clear picture of system interactions and workflows, ensuring developers and evaluators share a common understanding of the system's dynamic behaviour.

Upon implementation, SWMS will measurably streamline administrative workload, deliver transparent real-time academic data to students and parents, and reflect the commitment of the Department of Information Technology, The Islamia University of Bahawalpur to technology-driven academic excellence.

Prepared by	Submitted to	Date
Nageena Hakam Roll No: 4007 Dept. of IT, IUB	Sir Asad Ullah Faculty of Computing The Islamia University of Bahawalpur	June 10, 2026 Version: 1.0

Appendix A — API Endpoint Reference

The following table documents the key RESTful API endpoints exposed by the SWMS Laravel backend. All endpoints are prefixed with **/api/v1/**. Authentication requires a valid JWT Bearer token in the Authorization header or Secure HttpOnly cookie.

Method	Endpoint	Auth Role	Description
POST	/auth/login	Public	Authenticate user, return JWT token
POST	/auth/logout	Any	Invalidate current token
GET	/students	Admin	List all students (paginated)
POST	/students	Admin	Create new student record
GET	/students/{id}	Admin/Student	Get student by ID
PUT	/students/{id}	Admin	Update student record
DELETE	/students/{id}	Admin	Soft-delete student record
GET	/results/{student_id}	Admin/Student	Get all results for a student
POST	/results/upload	Admin/Teacher	Bulk upload results via CSV
GET	/attendance/{student_id}	Admin/Student	Get attendance records
POST	/attendance	Teacher	Mark attendance for a class
GET	/notices	Any	List active notices
POST	/notices	Admin/Teacher	Create a new notice
GET	/reports/result/{student_id}	Admin/Student	Download PDF result card
GET	/reports/attendance	Admin	Download attendance report (PDF/CSV)

Appendix B — References & Bibliography

[1] IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE, 2009.

[2] Laravel Documentation (v10.x). The PHP Framework for Web Artisans. https://laravel.com/docs/10.x

[3] MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[4] Redis Documentation. Redis Ltd. https://redis.io/docs/

[5] OWASP Top Ten — 2021 Edition. Open Web Application Security Project. https://owasp.org/Top10/

[6] Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.3/

[7] PHP 8.2 Release Notes. https://www.php.net/releases/8.2/

[8] Nginx Documentation. https://nginx.org/en/docs/

[9] JSON Web Tokens (JWT) — RFC 7519. IETF. https://tools.ietf.org/html/rfc7519

[10] WCAG 2.1 Guidelines. W3C Web Accessibility Initiative. https://www.w3.org/TR/WCAG21/

[11] PHPUnit Documentation. Sebastian Bergmann. https://phpunit.de/documentation.html

[12] Docker Documentation. Docker Inc. https://docs.docker.com/

End of Document | Nageena Hakam | Roll No: 4007 | Dept. of IT, IUB | 2026